

Planejamento Técnico e Arquitetural - Projeto Integrar Platform (V1)

Documento de referência para a arquitetura, escopo técnico e ordem de desenvolvimento da V1.

O Integrar Platform é uma aplicação web institucional desenvolvida para apoiar a divulgação e a captação de interessados para o grupo terapêutico Integrar. O sistema foi projetado sob três pilares fundamentais: Custo Zero Permanente (Always Free), Consistência Absoluta de Dados e Segurança em Conformidade com a LGPD.

1. Topologia de Rede e Fluxo de Dados (Síncrono e Confiável)

A arquitetura adota um fluxo linear sequencial na V1 para mitigar riscos de perda de dados por falhas de integração assíncronas sem a necessidade de mecanismos complexos de mensageria.

- Acesso e Entrega: O visitante acessa a Landing Page. O frontend (HTML/React/Animações) é distribuído globalmente através da infraestrutura da Cloudflare.
- Envio do Formulário: O visitante preenche os dados. O navegador dispara uma requisição HTTP POST segura (HTTPS) diretamente para a API em Go hospedada na Render.
- Pipeline do Backend (Go + Fiber): O middleware de segurança restringe quais origens podem chamar a API a partir do navegador (CORS), aplica controle de taxa de requisições, valida os dados, persiste no banco gerenciado, sincroniza com Google Sheets, aciona a Resend e retorna HTTP 200 OK ao frontend.

2. Escopo da Versão 1

Incluído

- Landing Page institucional responsiva com animações.
- Formulário de interesse (captação de leads).
- API REST funcional desenvolvida em Go.
- Banco de dados PostgreSQL gerenciado.
- Envio de e-mail transacional via API.
- Integração com a API do Google Sheets.
- Validação estrita de dados no backend.
- Caixa de consentimento explícito de privacidade (LGPD).
- Deploy automatizado em nuvem (CI/CD integrado ao Git).

Fora da V1 (Evoluções Futuras)

- Painel administrativo integrado.
- Autenticação e controle de sessões (Login/JWT).
- Goroutines para processamento assíncrono puro.
- Workers, filas de mensageria ou retry automático de APIs externas.
- Orquestração com Kubernetes ou provisionamento com Terraform.
- Gerenciamento e configuração manual de Máquinas Virtuais (VM Linux).

3. Stack Tecnológica

Frontend

- React + TypeScript + Vite: Interface SPA de alta performance com tipagem estática.
- Tailwind CSS: Estilização utilitária e responsiva.
- TanStack Router: Gerenciamento de rotas e estados de navegação de forma Type-Safe.
- Framer Motion: Biblioteca para controle e execução de animações fluidas na página.
- Cloudflare Pages: Hospedagem do frontend com HTTPS ativo e tráfego ilimitado de graça.

Backend

- Go (Golang): Alta velocidade de execução e baixo consumo de memória.
- Fiber: Framework web inspirado no Express com roteamento de alta performance.
- GORM: Padronização das interações com o banco, reduzindo o risco de SQL Injection ao parametrizar consultas quando usado corretamente.
- godotenv: Carregamento de credenciais locais.

Infraestrutura e APIs de Terceiros

- Neon / Supabase: PostgreSQL gratuito gerenciado.
- Resend API: Provedor de e-mails transacionais.
- Google Sheets API: Persistência secundária para espelhamento operacional dos contatos coletados.
- Render: Hospedagem gratuita da API em Go com deploy automatizado via GitHub.

4. Estrutura do Backend (Agrupamento por Contexto)

O projeto adota agrupamento por contexto de negócio (Feature Grouping), evitando excesso de camadas horizontais burocráticas.

```
api/
├── cmd/
│   └── api/
│       └── main.go           # Ponto de entrada da aplicação (Bootstrap).
├── internal/
│   ├── app/                 # Inicialização do Fiber, middlewares e rotas globais.
│   ├── config/              # Injeção e carregamento seguro do arquivo .env.
│   ├── database/            # Pool de conexões do GORM com PostgreSQL.
│   └── leads/                # Contexto do formulário isolado em um pacote.
│       ├── domain.go        # Struct do Lead e mapeamento do banco.
│       ├── handler.go       # Controlador HTTP do Fiber e validações de payload.
│       ├── repository.go    # Persistência interna do pacote (GORM).
│       └── service.go        # Chamadas para Google Sheets e Resend.
├── .env.example             # Modelo vazio das configurações necessárias.
├── go.mod                   # Gerenciador de módulos e dependências do Go.
└── go.sum                   # Checksums de segurança das dependências.
```

5. Modelo Inicial de Dados (Lead)

A entidade principal do sistema representará o visitante interessado e conterá os seguintes campos:

- ID - Chave primária identificadora.
- Name - Nome completo do interessado.
- Whatsapp - Número de contato principal.
- Age - Idade informada pelo visitante.
- Message - Texto/comentário opcional enviado.
- PrivacyConsent - Aceite obrigatório dos termos.
- CreatedAt - Data e hora exata da criação do registro.

6. Segurança e Privacidade (LGPD)

- HTTPS Obrigatório: Ativado no frontend e backend para cifrar dados em trânsito.

- Segregação de Credenciais: Chaves e senhas fora do código-fonte, usando variáveis de ambiente. O arquivo real .env é ignorado pelo Git.
- Restrição de Origem (CORS): Limita quais domínios web estão autorizados a consumir a API através do navegador.
- Controle de Abuso (Rate Limiting): Limita requisições por IP para dificultar spam no formulário.
- Terminologia Adequada: Entradas tratadas como Visitantes ou Interessados. O vínculo clínico ocorre somente após o início real do tratamento.

7. Variáveis de Ambiente Necessárias

```
PORT=8080
DATABASE_URL=postgres://usuario:senha@host:5432/banco?sslmode=require
RESEND_API_KEY=re_123456789
RESEND_EMAIL_TO=psicologo@email.com
GOOGLE_SHEET_ID=1a2b3c4d5e6f7g8h9i0j
GOOGLE_SERVICE_ACCOUNT_JSON={"type": "service_account", ...}
```

8. Ordem Sequencial de Desenvolvimento

Fase 1 - Base da API

- Criar a estrutura física de pastas.
- Inicializar o módulo Go (go mod init).
- Criar o pacote config/ para leitura de .env.
- Configurar a conexão com o PostgreSQL no pacote database/.
- Estruturar o inicializador central do Fiber em internal/app/.
- Criar uma rota básica de teste /health.

Fase 2 - Contexto de Leads

- Definir a struct no arquivo leads/domain.go.
- Criar o leads/repository.go para realizar o db.Create usando o GORM.
- Desenvolver o leads/handler.go para processar o JSON recebido.
- Registrar a rota POST /api/v1/leads no inicializador do app.
- Validar campos obrigatórios antes de salvar na base de dados.

Fase 3 - Integrações de Terceiros

- Criar as funções de chamada HTTP no arquivo leads/service.go.
- Integrar o SDK do Resend para envio do e-mail notificacional.
- Integrar a API do Google Sheets para registrar o lead na planilha.
- Configurar o fluxo sequencial síncrono no handler, adicionando logs de erro em caso de falha nas APIs externas.

Fase 4 - Polimento e Segurança

- Ativar e testar as configurações de CORS estrito.
- Configurar o middleware de Rate Limiting do Fiber.
- Tratar mensagens de erro da API para não expor detalhes do banco ao cliente.
- Garantir que chaves privadas estejam fora do histórico do Git.

Fase 5 - Publicação (Deploy)

- Publicar o frontend no Cloudflare Pages.
- Publicar a API Go na Render.
- Configurar todas as Environment Variables nos painéis web da Render e Cloudflare.
- Executar testes de ponta a ponta no ambiente de produção.